SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Design Task

Code of Conduct:

I Sowmiya. D certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sowmiya. D

Annexure A

Design Task

1. **Hybrid AI Recommendation Engine Design**
   Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. **Smart Disaster Detection System**
   Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework**
   Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. **Autonomous Supply Chain Optimization System**
   Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

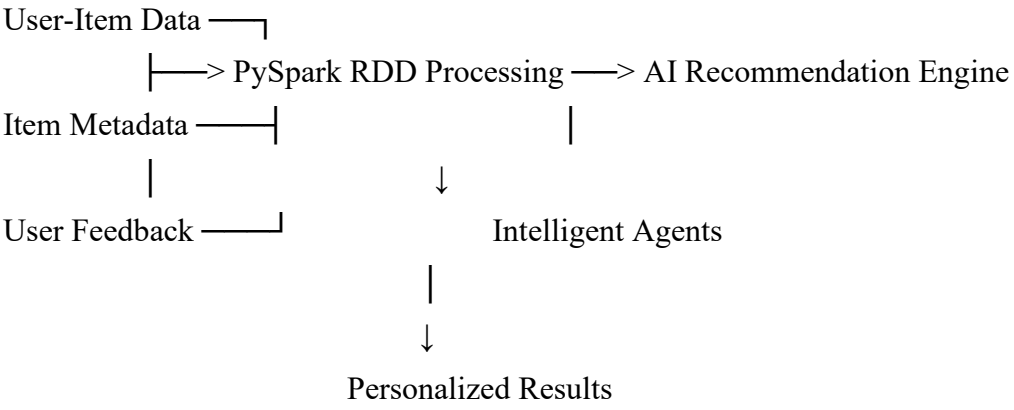
5. **Personalized Learning Analytics Platform**
   Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

1. Hybrid AI Recommendation Engine

Objective

Build a recommendation system that combines content-based filtering and collaborative filtering to recommend relevant products, movies, courses, or services to users.

System Architecture



PySpark RDD Processing

User-item interactions can be represented as:

(user_id, item_id, rating)

Item metadata can contain:

(item_id, category, keywords, description)

RDD transformations can process millions of interactions in parallel.

```
ratings = sc.textFile("ratings.csv")
user_items = ratings.map(
   lambda x: x.split(",")
).map(
   lambda x: (x[0], (x[1], float(x[2])))
)


high_rated = user_items.filter(
   lambda x: x[1][1] >= 4
)


items_by_user = high_rated.groupByKey()
```

Content-Based Filtering

The system creates a feature vector for every item using attributes such as:

- Category
- Keywords

- Description
- Price
- Genre

The user's profile is created from previously preferred items. Similarity measures such as cosine similarity can then identify suitable items.

Collaborative Filtering

Collaborative filtering finds users with similar behavior.

For example:

User A → Item 1, Item 2, Item 4

User B → Item 1, Item 2, Item 5

Because A and B have similar preferences, Item 5 can be recommended to A.

A distributed implementation can use user/item pair similarities or an algorithm such as distributed matrix factorization.

Hybrid Recommendation

The final score can combine both methods:

$$Score(u,i) = \alpha CB(u,i) + (1-\alpha)CF(u,i)$$

where:

- CB = content-based score
- CF = collaborative score
- α = weighting factor

Intelligent Agents

Different agents can perform specialized tasks:

- User Profile Agent: monitors clicks, ratings, searches, and purchases.
- Recommendation Agent: generates and ranks recommendations.
- Feedback Agent: analyzes likes, dislikes, skips, and purchases.
- Adaptation Agent: dynamically changes recommendation weights.
- Exploration Agent: introduces new items to avoid over-personalization.

When a user's behavior changes, agents update the profile and recommendations without rebuilding the entire system.

Advantages

- Handles millions of users and items.
- RDDs provide distributed and fault-tolerant processing.
- Hybrid filtering reduces the weaknesses of either method alone.
- Intelligent agents enable continuous personalization.

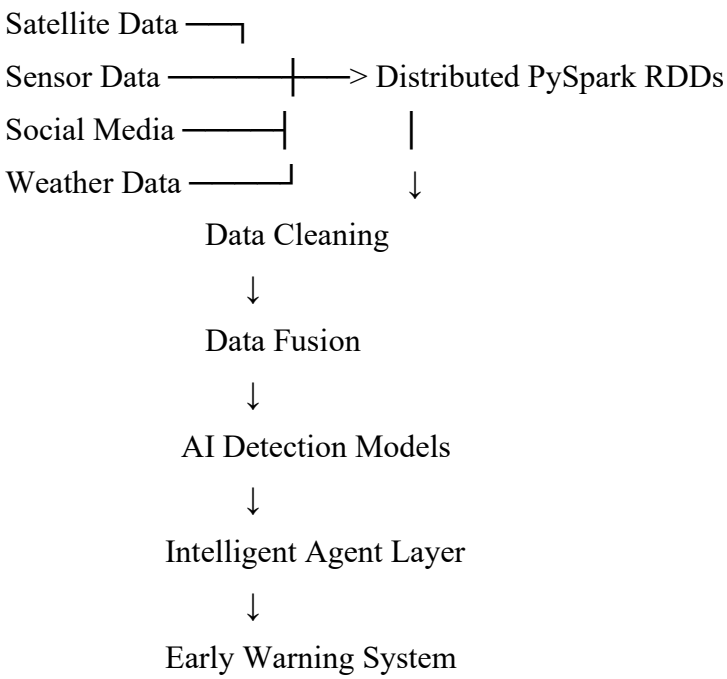
---

2. Smart Disaster Detection System

Objective

Develop a distributed AI system that detects floods, earthquakes, landslides, and other disasters by combining satellite imagery, IoT sensors, weather information, and social-media reports.

System Architecture



RDD Processing

Sensor data can be represented as:

(sensor_id, timestamp, location, measurement)

RDD operations can remove invalid readings:

```
sensors = sc.textFile("sensor_data.csv")


clean_data = sensors.map(
   lambda x: x.split(",")
).filter(
   lambda x: float(x[3]) >= 0
)
```

Data can then be grouped by geographic region:

```
regional_data = clean_data.map(
   lambda x: (x[2], x)
).groupByKey()
```

This allows thousands or millions of sensor readings to be processed across a cluster.

Multi-Source Data Fusion

The system combines:

Satellite data

- Water expansion

- Ground displacement
- Cloud/weather patterns
- Dam or river conditions

Sensor data

- Water level
- Rainfall
- Seismic activity
- Temperature and pressure

Social media

- Location-tagged reports
- Emergency keywords
- Images and videos

A fusion engine assigns confidence scores to observations from different sources.

For example:

$$DisasterScore = w_1(Satellite) + w_2(Sensor) + w_3(SocialMedia)$$

A high combined score can trigger an alert.

AI Detection

Possible models include:

- Classification for disaster/no-disaster prediction
- Clustering for unusual sensor behavior
- Anomaly detection for earthquakes or sudden water-level increases
- Computer vision for satellite-image analysis
- NLP for disaster-related social-media posts

Intelligent Agents

- Sensor Agent: continuously monitors IoT sensors.
- Satellite Agent: analyzes new satellite observations.
- Social Media Agent: identifies emergency-related posts.
- Fusion Agent: combines evidence from different sources.
- Decision Agent: determines disaster severity.
- Warning Agent: sends warnings to authorities and affected populations.
- Response Agent: recommends evacuation routes, shelters, and resource allocation.

Agents communicate their observations and confidence levels to coordinate a response.

Advantages

- Early detection from multiple independent sources.
- Distributed processing handles huge satellite and sensor datasets.
- Data fusion reduces false alarms.

- Intelligent agents enable autonomous and coordinated response.

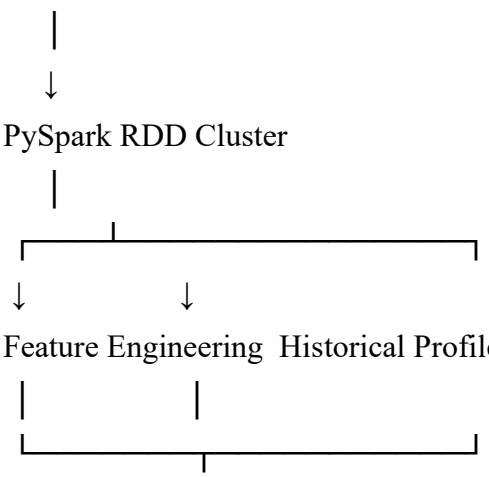
---

3. AI-Based Fraud Detection Framework

Objective

Create a scalable fraud-detection platform that analyzes millions of financial transactions and identifies suspicious behavior.

Architecture



RDD-Based Processing

A transaction might contain:

(transaction_id, user_id, amount, location, timestamp, merchant)

Example:

```
transactions = sc.textFile("transactions.csv")

tx = transactions.map(
    lambda x: x.split(",")
)

large_tx = tx.filter(
```

```
    lambda x: float(x[2]) > 10000
)


by_user = large_tx.map(
    lambda x: (x[1], x)
).groupByKey()
```

RDD transformations can calculate:

- Average transaction amount
- Transaction frequency
- Geographic changes
- Time-based behavior
- Merchant patterns
- Device patterns

Distributed Anomaly Detection

A transaction can receive an anomaly score:

$$A(x) = \sum_i w_i \, f_i(x)$$

where features may include:

- Unusually high amount
- Unusual location
- Unusual transaction time
- Rapid repeated transactions
- New device
- Abnormal purchasing pattern

Distributed clustering can also identify normal transaction groups. Transactions far from these groups are potential anomalies.

Intelligent Agents

Transaction Monitoring Agent
Checks incoming transactions.

Behavior Agent
Maintains a user's normal behavioral profile.

Pattern Agent
Searches for coordinated fraud involving multiple accounts.

Risk Agent
Combines anomaly scores into an overall fraud probability.

Alert Agent
Triggers an investigation when risk exceeds a threshold.

Learning Agent
Uses confirmed fraud/non-fraud decisions to improve future detection.

Example

Suppose a customer normally spends ₹2,000–₹5,000 in Chennai. Suddenly:

02:10 AM → ₹80,000 → Foreign location → New device

Multiple agents identify the abnormal behavior. The risk agent assigns a high score and the alert agent can request additional verification or temporarily flag the transaction.

Advantages

- Scales to billions of transactions.
- Distributed RDD operations reduce processing time.
- Combines statistical and AI-based detection.
- Agent feedback continuously improves the system.

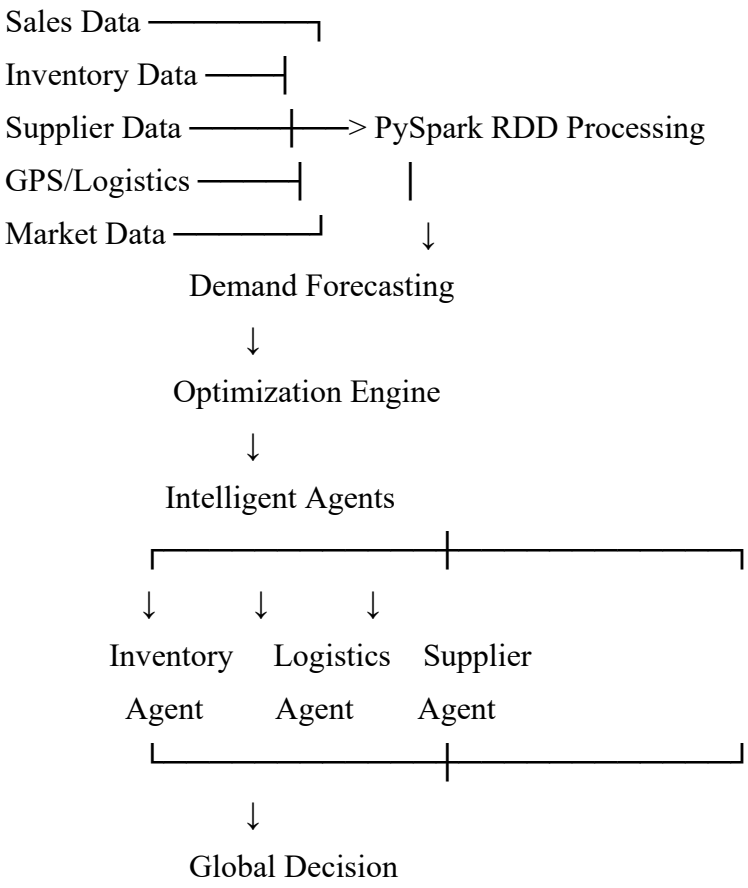
---

4. Autonomous Supply Chain Optimization System

Objective

Develop an AI system that optimizes inventory, transportation, warehouse operations, and demand forecasting using distributed PySpark processing and intelligent agents.

Architecture



RDD Transformations

Sales records can be represented as:

(product_id, date, quantity, location)

For example:

```
sales = sc.textFile("sales.csv")

records = sales.map(
   lambda x: x.split(",")
)

product_sales = records.map(
   lambda x: (x[0], int(x[2]))
)

total_sales = product_sales.reduceByKey(
   lambda a, b: a + b
)
```

map, filter, reduceByKey, groupByKey, and join can process large datasets across multiple machines.

Demand Forecasting

Historical sales are used to predict future demand.

The system can consider:

- Previous sales
- Seasonality
- Promotions
- Holidays
- Location
- Weather
- Market trends

Forecast:

$$D_{t+1} = f(D_t, D_{t-1}, S, e, asonPromotionMarket)$$

The predicted demand determines the appropriate inventory level.

Intelligent Agents

Inventory Agent

- Monitors stock.
- Predicts shortages.
- Generates replenishment decisions.

Logistics Agent

- Selects transportation routes.

- Considers distance, delivery time, and cost.

Warehouse Agent

- Optimizes storage and picking.

Supplier Agent

- Evaluates supplier price, reliability, and delivery time.

Demand Agent

- Continuously updates demand forecasts.

Coordination Agent

- Resolves conflicts between local agents and selects an overall efficient strategy.

Decentralized Decision Making

For example:

Inventory Agent → Stock is low

↓

Demand Agent → Demand expected to increase

↓

Supplier Agent → Supplier A has lowest cost

↓

Logistics Agent → Route B has shortest delivery time

↓

Coordination Agent → Place order with Supplier A using Route B

The objective can be expressed as:

$$Minimize\left(C_{inventory} + C_{transport} + C_{shortage} + C_{warehouse}\right)$$

Advantages

- Reduces excess inventory.
- Prevents stock-outs.
- Optimizes transportation.
- Supports decentralized autonomous decisions.
- Adapts to changing demand and supply conditions.

---

5. Personalized Learning Analytics Platform

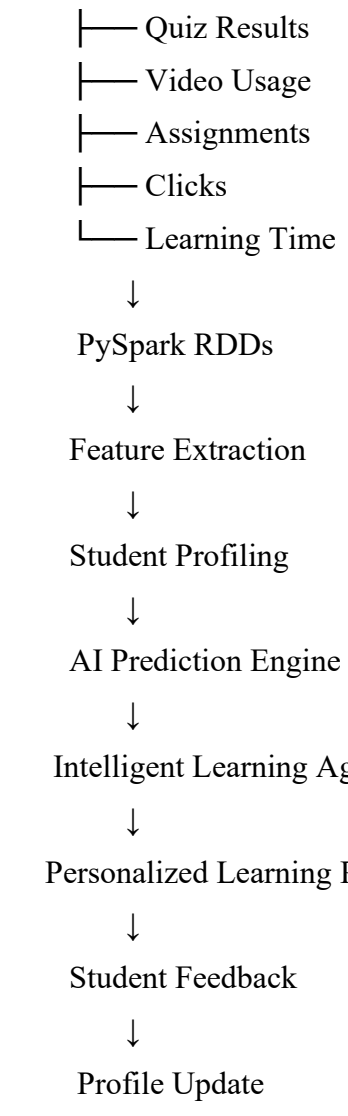
Objective

Build an AI-driven educational platform that analyzes student behavior and automatically adapts learning content for thousands of learners.

Architecture

Student Interactions

|



RDD Processing

Learning events can be represented as:

(student_id, course_id, activity, score, time_spent)

```
events = sc.textFile("learning_events.csv")

data = events.map(
    lambda x: x.split(",")
)

student_scores = data.map(
    lambda x: (x[0], float(x[3]))
)

average_scores = student_scores \
    .groupByKey() \
```

```
    .mapValues(lambda scores: sum(scores) / len(scores))
```

RDDs can analyze the activities of thousands or millions of learners in parallel.

Student Behavior Analysis

The platform can calculate:

- Quiz accuracy
- Completion rate
- Time spent per lesson
- Number of attempts
- Frequently incorrect concepts
- Learning speed
- Dropout risk
- Preferred learning formats

A learner profile might look like:

Student 101

Mathematics: Strong

Programming: Medium

Statistics: Weak

Learning speed: Fast

Preferred content: Video + Examples

Adaptive Learning

The AI recommends content according to student performance.

For example:

Low quiz score

↓

Identify weak concept

↓

Recommend simpler explanation

↓

Give example

↓

Practice questions

↓

Re-test

↓

Update student profile

Intelligent Agents

Student Monitoring Agent
Tracks learning behavior.

Assessment Agent
Analyzes test and assignment performance.

Content Agent
Selects appropriate lessons and exercises.

Feedback Agent
Provides immediate feedback.

Difficulty Agent
Raises or lowers question difficulty.

Motivation Agent
Detects disengagement and recommends interventions.

Teacher Agent
Provides instructors with summaries of students who require attention.

Real-Time Personalization

If a student repeatedly fails questions about a particular concept, the system can immediately:

1. Detect the knowledge gap.
2. Recommend prerequisite material.
3. Reduce question difficulty.
4. Provide additional examples.
5. Reassess the student.
6. Increase difficulty after improvement.

Advantages

- Supports thousands of students simultaneously.
- RDDs enable distributed learning analytics.
- Intelligent agents provide individualized learning.
- Continuous feedback improves recommendations.
- Teachers receive data-driven insights.

Overall Comparison

System	Main Data	AI Technique	Key Intelligent Agents	Main Benefit
Recommendation	User-item interactions	Content + Collaborative Filtering	Profile, Recommendation, Feedback	Personalization
Disaster Detection	Sensors, satellite, social media	Classification + Anomaly Detection	Sensor, Fusion, Warning	Early warning
Fraud Detection	Financial transactions	Anomaly Detection + Classification	Monitoring, Risk, Alert	Fraud prevention

Supply Chain	Sales, inventory, logistics	Forecasting + Optimization	Inventory, Logistics, Supplier	Cost reduction
Learning Analytics	Student activities	Prediction + Personalization	Student, Content, Feedback	Adaptive learning

Common PySpark + Intelligent Agent Model

All five systems follow the same fundamental pattern:

Large-scale data → RDD-based distributed processing → feature extraction → AI analysis → intelligent agents → decision/recommendation → user/system feedback → continuous adaptation.

PySpark provides the scalable data-processing layer, while AI models provide prediction or anomaly detection, and intelligent agents provide the autonomous decision-making and adaptation layer.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation